

Comparative Analysis of Reinforcement Learning Algorithms for Autonomous Maze Navigation

Sandesh Sampath Kumar

Foundations of Artificial Intelligence, Northeastern University, April 2026

1. Introduction

Maze navigation is a classic benchmark problem in artificial intelligence, and for good reason. It captures the core structure of sequential decision-making: an agent must find a path from a start location to a goal, navigate around obstacles, and do so efficiently. The task is simple enough to be tractable, yet rich enough to expose real differences in how learning algorithms acquire, store, and reuse knowledge. These qualities make it a useful setting for comparing reinforcement learning approaches.

In reinforcement learning, an agent interacts with its environment step by step, collecting reward signals that guide it toward better behavior over time [2]. Early approaches like Q-Learning capture this by maintaining a table of state-action values and updating them incrementally after each step [2]. Osmanković and Konjicija [1] applied Q-Learning directly to maze problems and confirmed that it converges reliably on a fixed maze. They also identified the key limitation: the Q-table is entirely tied to one specific maze layout, and switching to a different maze wipes out everything the agent has learned. This makes the tabular approach a poor fit for any setting that requires navigating multiple distinct environments.

Deep Q-Networks sidestep this by replacing the lookup table with a neural network that takes the full maze grid as input [2]. The agent can then distinguish between maze layouts and in principle learn navigation strategies that generalize. Two refinements that proved important in practice are Double DQN, which corrects a tendency to overestimate Q-values [2], and the Dueling Network architecture, which separately estimates how good a state is versus how much each action improves on that baseline [2]. Both extensions lead to more stable and accurate value estimates.

Policy gradient methods take a different route by optimizing the policy directly rather than through a value function. Proximal Policy Optimization (PPO) is one of the most widely used algorithms in this family [3]. It updates the policy using gradient ascent while constraining each update with a clipping mechanism that prevents any single step from changing the policy too dramatically. PPO pairs this with Generalized Advantage Estimation (GAE) to reduce the variance in gradient estimates [4, 5]. Unlike DQN, PPO is an on-policy algorithm, meaning it can only learn from experience collected under its current policy. This creates a real challenge when training across multiple non-stationary environments, as stale experience from a replay buffer is not available to smooth out distribution shifts between mazes.

This project implements all three algorithms in a shared custom environment and compares them side by side. The maze environment was built from scratch using recursive depth-first search with configurable wall removal to produce mazes of varying complexity. Each algorithm runs under two experimental conditions: a single fixed maze to measure raw convergence, and a set of five distinct mazes to test generalization. The following sections describe the environment and algorithms in detail, present the training results for all six configurations, and discuss what the outcomes reveal about each approach.

2. Methods

2.1 Maze Environment

The maze environment was built as a Gymnasium-compatible class [6], meaning it follows the standard reset-step-render interface and can be paired with any RL library that expects this API. Mazes are generated procedurally using recursive DFS backtracking. This algorithm carves a path through the grid cell by cell, backtracking whenever it hits a dead end, and guarantees that every cell in the maze is reachable from the start. After the base maze is generated, 30% of interior walls are removed at random to create loops and branching paths.

Each cell in the grid is encoded as an integer: 0 for an open path, 1 for a wall, 2 for the start position at row 1 column 1, 3 for the exit placed at the cell farthest from the start by shortest-path distance, and 4 for trap cells. Traps look like open paths but incur a penalty when stepped on. In single-maze experiments, 10% of open cells are randomly set as traps. For Q-Learning, the agent observes only its current (row, column) position. DQN and PPO receive a richer input: the entire maze grid, flattened and normalized by dividing by 4, concatenated with the agent's row and column coordinates normalized by the maze height and width respectively. This produces a 123-dimensional vector for an 11x11 maze.

The reward structure is designed to push the agent toward short, direct paths. Reaching the exit earns +10.0 points. Each step taken costs -0.1, so unnecessary wandering is penalized. Stepping on a trap costs -1.0, and bumping into a wall costs -0.3 with no movement. Episodes end when the agent reaches the exit or when 500 steps pass without success. Q-Learningm DQN and PPO used 11x11 mazes.

2.2 Q-Learning

Q-Learning serves as the baseline in this study. The agent stores its learned knowledge in a lookup table, implemented as a Python defaultdict that maps each (row, column) position to an array of four Q-values, one per action. To decide what to do at each step, the agent uses an epsilon-greedy policy: with probability epsilon it picks a random action to explore, and otherwise it picks the action with the highest Q-value for its current position. Epsilon starts at 1.0 and decays by a factor of 0.995 after each episode, reaching a floor of 0.01. Q-values are updated using the standard temporal-difference rule: $Q(s,a)$ gets nudged toward the observed reward plus a discounted estimate of the best future value, with a learning rate of 0.1 and a discount factor of 0.99.

For the single-maze experiment, training ran for 10,000 episodes on a fixed 11x11 maze seeded at 42. The multi-maze experiment trained the same agent sequentially across five mazes with seeds 42, 99, 7, 123, and 256, spending 2,000 episodes on each. The agent was warm-started from the single-maze weights before beginning multi-maze training, which lets us observe directly how well prior knowledge carries over and how quickly it gets displaced. After training, the agent was evaluated for 100 greedy episodes with epsilon set to zero.

2.3 Deep Q-Network (DQN)

The DQN agent builds on the basic Q-Learning idea by replacing the Q-table with a neural network. Two well-known improvements are included: Double DQN to reduce Q-value overestimation [2], and the Dueling Network architecture to separately estimate state value and action advantages [2]. The network takes the 123-dimensional state vector as input and passes it through two fully-connected layers of widths 512 and 256, both using Layer Normalization and ELU activations. The output splits into a value head and an advantage head, which are

combined using the standard Dueling formula: $Q(s,a) = V(s) + A(s,a) - \text{mean}(A(s,a))$. A target network, synchronized every 500 gradient steps, provides stable regression targets and prevents the training signal from chasing a moving estimate.

Training uses the Adam optimizer [7] with a learning rate of $5e-4$ and Huber loss. Experience is stored in a circular replay buffer with a capacity of 500,000 transitions. Learning does not begin until 2,000 transitions have been collected, and one gradient step is taken every four environment steps using mini-batches of 256. For the single maze, training ran for 3,000 episodes on a fixed 11x11 maze. For the multi-maze condition, training used a cyclic schedule that rotated through all five mazes every 100 episodes, completing six full rotations across 3,000 total episodes. Cycling through mazes frequently keeps transitions from all five environments in the buffer at all times, which naturally counteracts the forgetting problem.

2.4 Proximal Policy Optimization (PPO)

The PPO agent uses an Actor-Critic architecture with the same 123-dimensional state encoding as DQN. The shared network trunk consists of two fully-connected layers with widths 256 and 128, both using Tanh activations. Layer Normalization was deliberately left out of the trunk. When normalization is applied inside a shared policy network, it rescales activation magnitudes on each forward pass, which interferes with the policy ratio that the PPO clipping mechanism relies on to bound updates. Removing it produced noticeably more stable training. All weights are initialized orthogonally, which is standard practice for PPO. The actor head uses a small initialization gain of 0.01 so the policy starts close to uniform across all actions, and the critic head adds one small hidden layer before predicting a scalar state value.

The training loop collects rollouts of 2,048 environment steps before each update. Advantages are computed using GAE with $\lambda = 0.95$ and $\gamma = 0.99$ [4, 5], then normalized across the rollout for stable gradient magnitudes. The PPO-Clip objective with $\epsilon = 0.2$ is then optimized over up to 10 epochs using mini-batches of 64 transitions. An approximate KL divergence check runs at the end of each epoch: if the mean of 0.5 times the squared log-ratio between the new and old policy exceeds 0.02, the remaining epochs are skipped. This acts as an adaptive brake that allows all 10 epochs when the policy is changing slowly and cuts updates short when the gradient is pushing the policy too hard, which is particularly useful in the noisy multi-maze setting. The critic is trained using mean squared error loss rather than Huber loss. The default Huber delta of 1.0 is too small for this environment, where episode returns can range from roughly -50 to +10. At that scale the Huber loss spends most of its time in the linear regime, giving the value network very weak gradient signal. The total loss combines the actor loss, a value loss weighted by 0.5, and an entropy bonus weighted by 0.05. Raising the entropy coefficient from 0.01 to 0.05 was necessary to slow down the rate at which the policy commits to specific actions, preventing the early collapse observed in prior runs. The Adam optimizer [7] runs at a learning rate of $2e-4$, and gradients are clipped to a maximum norm of 0.5.

Single-maze training ran for up to 600,000 environment steps on a fixed 11x11 maze. An early stopping mechanism saves the best checkpoint seen so far and restores it if the running success rate falls significantly below that peak. For multi-maze training, the agent was initialized from scratch rather than from the converged single-maze weights. That warm-start approach failed in early experiments: the single-maze policy had already committed strongly to patterns specific to one maze layout, with an entropy of around 0.4 nats out of a theoretical maximum of 1.39. When that policy encountered the four other mazes, whose grid layouts account for 98% of the input features, it produced confident but wrong actions at every step. The agent never stumbled onto the exit, never received a positive reward signal, and the policy gradient had nothing

meaningful to work with. Starting from random weights avoids this entirely: the initial policy is approximately uniform, so the agent naturally explores all five mazes and accumulates reward signals from the start. The multi-maze setup uses the same randomized per-episode maze selection and the same early stopping logic as the single maze run.

2.5 Evaluation

Each trained agent was evaluated over 100 episodes after training completed. Q-Learning and DQN used greedy action selection with epsilon set to zero. PPO sampled from its learned action distribution, since using argmax on a stochastic policy can lead to deterministic loops in some states. The three metrics recorded during training were cumulative episode reward, steps taken to reach the exit or the truncation limit, and success rate as a binary signal per episode. DQN additionally logged Huber training loss per update. PPO logged actor loss, MSE critic loss, and policy entropy per update. All training curves were smoothed with a 50-episode rolling average before plotting. Animated GIFs of the trained agent solving the maze were also generated using Matplotlib and Pillow to provide a qualitative view of the learned policy. The full software stack was Python 3, PyTorch [7] for neural network training, Gymnasium [6] for the environment interface, and NumPy for numerical computation.

3. Results

3.1 Q-Learning on a Single Maze

Q-Learning had no trouble with the single fixed 11x11 maze. Training started with the agent exploring almost randomly, accumulating large time penalties that pushed the cumulative reward down to around -100 in early episodes. As epsilon decayed and the Q-table filled in, the agent found shorter paths to the exit and the reward climbed steadily toward zero. By around episode 2,000, performance had effectively converged. The average steps to solve dropped from 500, the truncation limit, to a stable range of 50 to 80 steps. The solve rate crossed 90% by episode 1,500 and sat at or near 100% for the rest of training. Epsilon reached its minimum of 0.01 around episode 900, after which the agent was running almost entirely on what it had already learned.

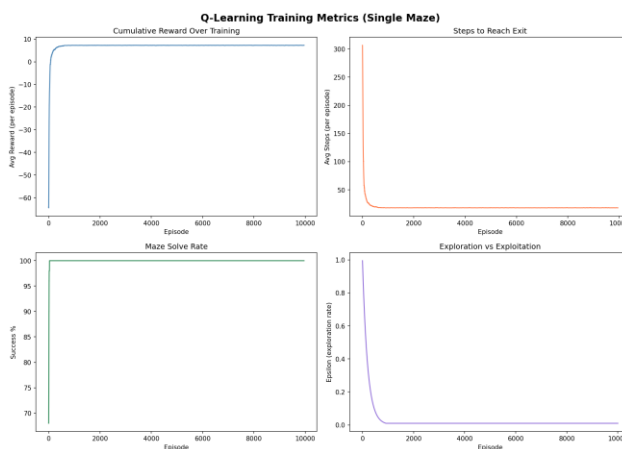


Figure 1. Q-Learning training metrics on a single maze (11x11, seed 42, 10,000 episodes). Top-left: cumulative reward. Top-right: steps to exit. Bottom-left: solve rate. Bottom-right: epsilon.

3.2 Q-Learning Across Multiple Mazes

The multi-maze results tell a different story. Every time training moved to a new maze, all three performance metrics collapsed back to their starting levels. The agent that had just learned to navigate maze 1 efficiently was essentially starting over on maze 2. Figure 2 shows this pattern repeating across all five mazes. Convergence on each individual maze was relatively quick, typically 500 to 700 episodes, partly because epsilon was already low when each new maze began. But there was no sign that learning one maze made the next one any easier. The agent did not transfer any navigational skill across environments.

This behavior is consistent with what Osmanković and Konjicija observed in [1]: the Q-table maps raw grid coordinates to values, and a position (row, column) in one maze bears no meaningful relationship to the same coordinates in a different maze. Values learned for maze 1 do not help in maze 2; they actively mislead the agent. Warm-starting from the single-maze model provided no benefit for the same reason. The coordinate system itself is non-transferable.

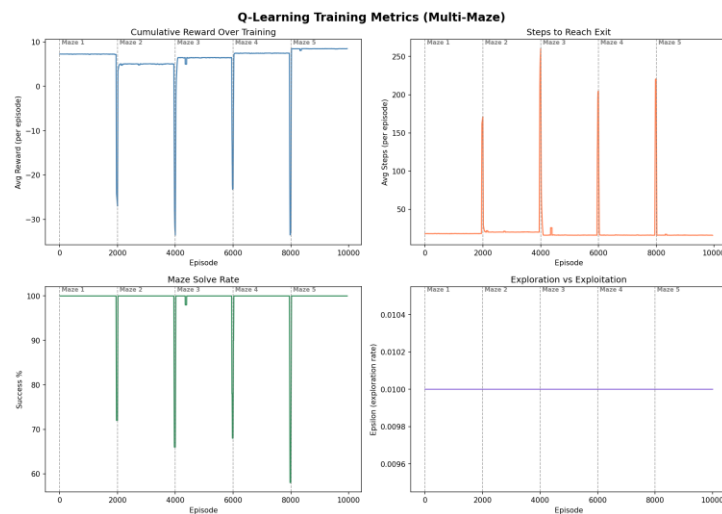


Figure 2. Q-Learning training metrics across five mazes (2,000 episodes each). Dashed vertical lines mark maze transitions. Each transition triggers a full performance collapse and relearning cycle.

3.3 DQN on a Single Maze

DQN converged smoothly on the single 11x11 maze over 3,000 episodes. Cumulative reward climbed from below -100 to near zero as the agent learned more efficient routes. Steps to exit fell from around 400 in early training to under 100 by the end. The solve rate crossed 90% around episode 800 and stayed there. The Huber training loss was high early on while the replay buffer was filling and the network was making large adjustments, then settled to near zero with occasional small spikes as the agent explored new parts of the maze. The slower epsilon decay of 0.999 per episode meant exploration stayed active longer than in Q-Learning, which gave the replay buffer broader coverage of the state space before the policy started to solidify.

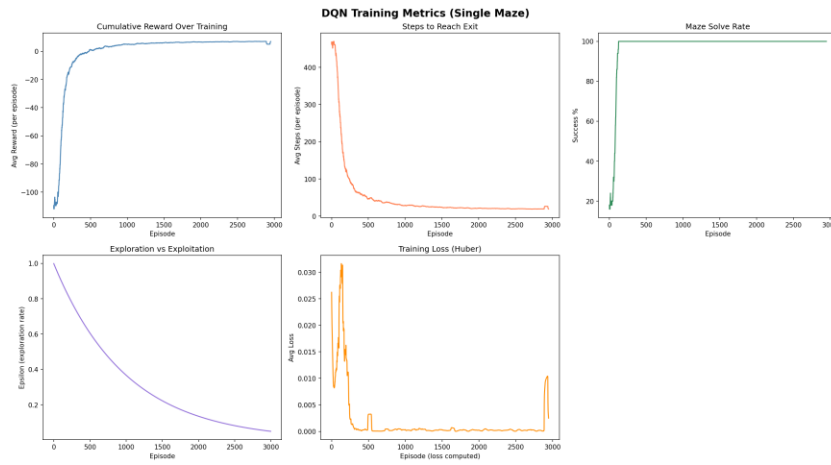


Figure 3. DQN training metrics on a single maze (11x11, seed 42, 3,000 episodes). Panels show cumulative reward, steps to exit, solve rate, epsilon, and Huber training loss.

3.4 DQN Across Multiple Mazes

DQN handled multi-maze training much better than Q-Learning. Looking at Figure 4, there are small dips in performance at each 100-episode cycle boundary, reflecting the brief readjustment period each time the agent returns to a maze it has not seen for 400 episodes. But the solve rate generally stayed above 80% throughout training and the overall reward trend was upward across all 3,000 episodes. The training loss followed the same shape as the single-maze case.

Two things explain this resilience. First, the state input includes the full maze grid, so the network can distinguish between mazes rather than treating every environment as the same coordinate space. Second, the replay buffer holds transitions from all five mazes at once. No single maze can dominate the training data, which naturally prevents the forgetting that derailed Q-Learning. The cyclic rotation schedule reinforces this by ensuring that every maze gets revisited every 500 episodes.

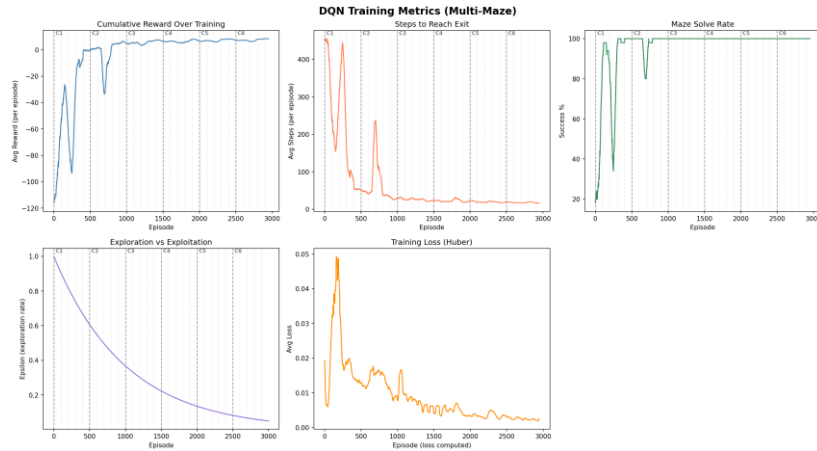


Figure 4. DQN training metrics across five mazes using a cyclic schedule (100 episodes per maze, 3,000 total episodes). Dashed vertical lines mark cycle boundaries.

Performance holds up across transitions, unlike Q-Learning.

3.5 PPO on a Single Maze

With the refined architecture, PPO performed well on the single fixed maze. Cumulative reward rose from around -60 in early episodes to near zero, and the average steps to solve fell from roughly 250 to about 50 by the time training converged. The solve rate reached close to 100% within about 1,000 episodes. There was a notable setback around episode 3,000: the solve rate dropped sharply and steps to exit spiked back up, suggesting the policy had destabilized. The early stopping mechanism caught this, restored the best saved checkpoint, and performance recovered immediately. This kind of transient collapse is a known risk in PPO when entropy drops too low too quickly, and the combination of a higher entropy coefficient and checkpoint restoration kept it from becoming a permanent problem.

The actor loss oscillated closely around zero throughout training, which is the expected behavior when policy updates are staying within the clipped region. The MSE critic loss started high at around 15, reflecting the value network learning from scratch, and trended down as the critic's predictions improved. Policy entropy decreased from about 1.39 nats at initialization to around 0.4 to 0.5 nats at convergence. The agent was becoming more decisive over time, which is what a well-trained policy should look like.

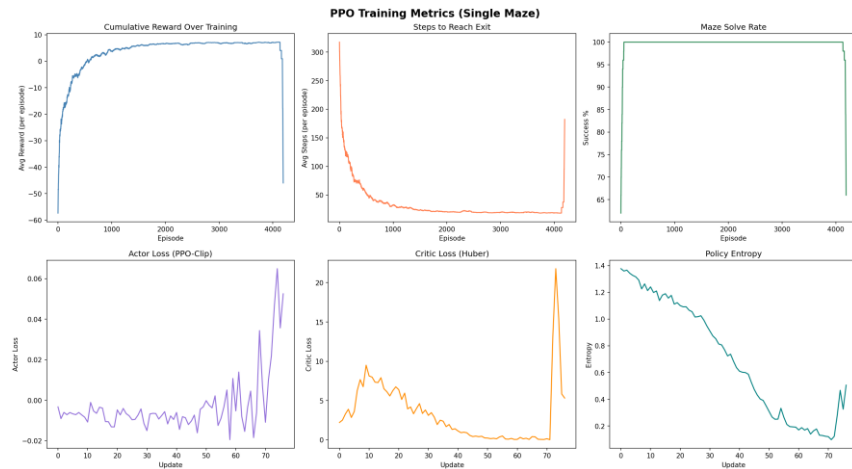


Figure 5. PPO training metrics on a single maze (11x11, seed 42, up to 600,000 timesteps). Top row: cumulative reward, steps to exit, solve rate. Bottom row: actor loss, MSE critic loss, and policy entropy per update. The collapse around episode 3,000 is recovered by checkpoint restoration.

3.6 PPO Across Multiple Mazes

The multi-maze PPO results represent the biggest turnaround in this project. An earlier version of the training setup, which warm-started from the single-maze model, produced a 0% solve rate across all 1,200 evaluation episodes. The agent was never reaching the exit, not even by accident. The core problem was that the converged policy had very low entropy and was deeply committed to navigation patterns specific to maze seed 42. When it encountered the four other maze grids, which make up 98% of the input features, it consistently made confident but wrong decisions. Without any successful episodes, there was no positive reward signal to drive learning.

Switching to a randomly initialized agent for multi-maze training fixed this completely. Figure 6 shows the result. Cumulative reward rose smoothly from around -40 to near zero over roughly 21,000 episodes, a much cleaner trajectory than earlier multi-maze PPO runs. Steps to exit fell from about 250 to around 50, matching the performance DQN achieved. The solve rate built up steadily and converged to near 100%, staying there with only small fluctuations caused by the random maze selection at episode resets. Policy entropy decreased along the same

path as the single-maze run, confirming that the agent was progressively building a real strategy rather than just maintaining a random policy out of necessity.

The KL divergence early stopping also played a meaningful role. On rollouts collected from a single maze the policy tends to update consistently across all mini-batches, and all 10 epochs typically completed. On noisier rollouts spanning multiple maze layouts with conflicting gradient directions, the KL check fired after two or three epochs and stopped further updates. This kept the policy from being pushed too far in any one direction during a single training step, which was the main failure mode in earlier runs that used a fixed epoch count.

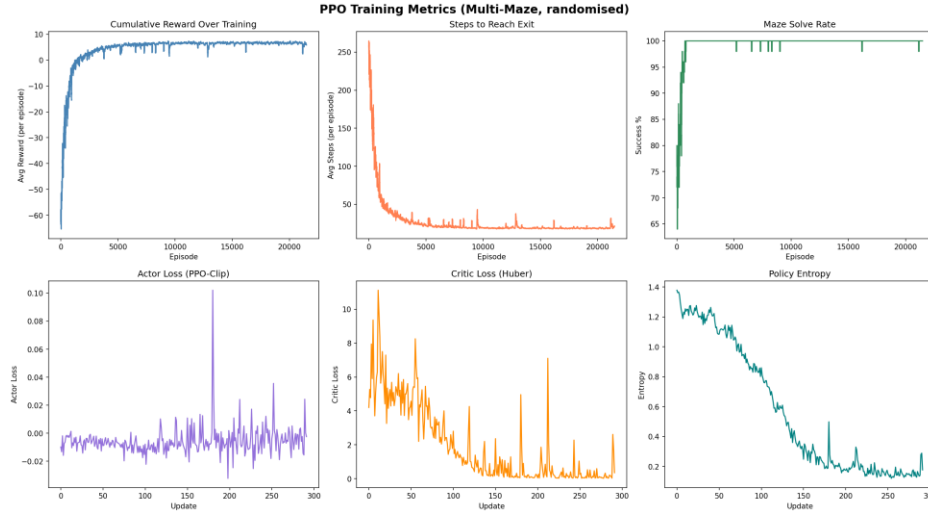


Figure 6. PPO training metrics across five mazes with random maze selection per episode and a fresh agent initialization (up to 600,000 timesteps). The agent converges to near 100% solve rate with smooth reward and step trajectories, in contrast to earlier configurations that achieved 0% throughout training.

4. Conclusions

The three algorithms tested here produced similar results on a single maze and very different results when asked to generalize across five. That gap is where the interesting lessons are.

Q-Learning works well for a fixed environment. It converged quickly on the 11x11 maze and the final policy was close to optimal. The problem is that everything it learns is tied to one specific maze layout. When training moves to a new maze, the Q-table entries from the previous maze actively conflict with what needs to be learned for the next one. This is not a flaw in the implementation; it is a fundamental property of the tabular representation. As Osmanković and Konjicija noted in [1], Q-Learning applied to mazes produces policies that are essentially maze-specific lookup tables with no ability to generalize. That said, Q-Learning has real advantages for single-environment tasks: it requires no neural network, it is easy to inspect and debug, and it converges fast.

DQN generalized better than the other two methods in this study. Encoding the maze grid as part of the state gave the network enough information to tell environments apart and learn separate strategies for each. The replay buffer served an additional purpose beyond just breaking temporal correlations: by holding transitions from all five mazes simultaneously, it kept every environment in the training distribution throughout the run. The cyclic rotation schedule worked well with this design. The main cost is memory. A 500,000-transition

buffer takes meaningful storage, and the 2,000-transition warm-up period before learning begins introduces a delay that on-policy methods do not have.

PPO was the most sensitive algorithm to implementation choices. The changes that made a real difference were: removing Layer Normalization from the trunk to stabilize the policy ratio, switching to MSE critic loss to give the value network adequate gradient signal, raising the entropy coefficient to 0.05 to prevent the policy from committing too early, using adaptive epoch stopping via a KL threshold to prevent over-optimization on noisy multi-maze rollouts, and starting multi-maze training from random weights rather than a converged single-maze policy. Each of these changes addressed a specific failure mode. Without all of them together, multi-maze PPO produced zero successful episodes. With all of them, it converged to near-perfect performance. This outcome illustrates how on-policy algorithms can be competitive with off-policy ones when properly configured, but that the path to a good configuration is less forgiving.

There are several directions this work could grow in. Scaling the mazes significantly would test whether the grid-based state encoding continues to work or whether more compact representations become necessary. Limiting the observation to a small local window would require the agent to build a mental map of the maze as it explores, which is a harder problem that might favor recurrent architectures. Framing the multi-maze problem as meta-learning, where the goal is to adapt quickly to a new maze from just a few episodes of experience, would directly address the generalization challenge rather than working around it. There is also the question of whether a method that combines DQN's off-policy buffer with PPO's policy gradient updates could outperform both on the multi-maze task.

To summarize: tabular Q-Learning is fast and interpretable but cannot generalize. DQN handles multi-maze training well through replay and grid-aware state encoding. PPO can match DQN's multi-maze performance, but it needs more careful setup and is less forgiving of configuration mistakes. All three results point to the same underlying lesson: knowing which algorithm to use is only half the problem. Getting that algorithm configured correctly for the specific task at hand is the other half, and often the harder one.

References

- [1] D. Osmanković and S. Konjicija, "Implementation of Q-Learning algorithm for solving maze problem," in Proc. 34th Int. Convention MIPRO, Opatija, Croatia, May 2011.
- [2] L. Weng, "A (Long) Peek into Reinforcement Learning," Lil'Log, Jun. 2018. [Online]. Available: <https://lilianweng.github.io/posts/2018-02-19-rl-overview/>
- [3] OpenAI, "OpenAI Baselines: PPO," OpenAI Blog, Jul. 2017. [Online]. Available: <https://openai.com/research/openai-baselines-ppo>
- [4] L. Weng, "Policy Gradient Algorithms," Lil'Log, Apr. 2018. [Online]. Available: <https://lilianweng.github.io/posts/2018-04-08-policy-gradient/>
- [5] OpenAI, "Proximal Policy Optimization," OpenAI Spinning Up, 2018. [Online]. Available: <https://spinningup.openai.com/en/latest/algorithms/ppo.html>
- [6] Farama Foundation, "Gymnasium Documentation," 2023. [Online]. Available: <https://gymnasium.farama.org>
- [7] PyTorch Team, "PyTorch: An Open Source Machine Learning Framework," Meta AI, 2023. [Online]. Available: <https://pytorch.org>